

Reinforcement Learning for Optimal Game-Theoretic Strategies

Eshaan Kumar

Mentor: Alex Kokot (Ph.D. Candidate) | University of Washington, Statistics Department

Directed Reading Program | Spring 2026

1 Introduction

In this project, we aim to train a computer agent to play a two-player card game. Our goal is to use reinforcement learning to find an optimal strategy for a variant of Colonel Blotto and Goofspiel. The reinforcement agent gets rewarded for winning and penalized for losing, and gradually improves its decisions over time.

The card game involves hidden information, as you cannot see your opponent's hand, and decisions that play out across multiple rounds, where a move that looks good in round one can hurt you in round two or three. Considerations such as these increase the complexity of the game and lead to a difficult optimization task. The project was conducted as part of the UW Statistics Directed Reading Program under mentor Alex Kokot.

The main question was: can an RL agent learn strategies that work well against many different kinds of opponents, not just the one it trained against? We tried three different approaches, each building on what we learned from the last.

2 Environment and Problem Formulation

Each player has their own personal 13-card deck, with card values from Ace (1) through King (13). At the start of a game, each player draws 7 cards. Players then take turns either playing a card or passing (or if a player is out of cards in their current hand they automatically pass). When both players have passed, the round ends, and whoever played cards with a higher total value wins that round. Each player then draws 3 more cards, and the next round begins. The first player to win 2 rounds wins the game.

2.1 MDP Formulation and Action Masking

The game is formalized as a Markov Decision Process (MDP). At each step the agent receives a list of 50 numbers, the state space, describing the current game state. This includes: which cards are in its hand (13 slots, one per rank), which cards it has played this round (13 slots), which cards the opponent has played this round (13 slots), and 11 additional numbers covering game context such as the round number, how many rounds each player has won, who acts first, whether each player has passed, and each player's remaining deck size.

One limitation of this design is that the observation resets the played-card slots at the start of each new round, meaning the agent has no direct record of what was played in previous rounds. This is a form of partial observability, as knowing that the opponent played their King in round 1 would be useful information in round 2, but the current state space does not capture it. The agent can partially infer its own past plays by comparing its current hand to what it knows it has drawn, but the opponent's previous round history is fully unobserved. Expanding the state

space to include a running history of all cards played across rounds would be a natural improvement to the observation design.

The agent can choose from the 14 possible actions in its action space: pass, or play any one of the 13 card ranks. If the agent tries to play a card it does not have, that action is automatically blocked. The training system incorporates action masking by making those options effectively impossible before the agent chooses, so it can only ever pick from cards it actually holds.

2.2 Reward Structure

Due to the complex structure of the game, care is needed to construct a meaningful reward signal. If we reward the agent just for winning individual rounds, it learns to dump its best cards early to win round 1, which often means it loses the overall game. To fix this, we use a layered reward: a large reward (+5 / -5) for winning or losing the full game, a smaller reward (+0.1 / -0.1) for each individual round, and a tiny ongoing signal (+/-0.02 times the change in score difference) during each round.

2.3 Statistical Evaluation

Going first is a disadvantage, so all win rates are computed bidirectionally: the agent plays as Player 1 in half the games and Player 2 in the other half, and we average the two. Win rate is a Bernoulli probability, and there are explicit ways to construct these confidence intervals carefully. Our win rate estimates include 95% confidence intervals using the Wilson method, which is more reliable than a simple normal approximation when win rates are near 0% or 100%. Each evaluation uses 100 games per opponent.

3 Baseline Agents

We use three rule-based agents as benchmarks throughout the project.

RandomBot picks a random legal action every turn. If our agent cannot beat this consistently, it has not learned anything useful.

SmartBot always plays the smallest card that puts your running round total above the opponent's. If no card can do that, pass. In the final round, play everything. SmartBot is decent at winning individual rounds but does not think about the overall game.

BaselineBot is a more sophisticated rule-based agent. It makes smarter decisions by considering the whole game, not just the current round.

At each turn it receives the current hand, both players' running round scores, the round number, each player's win count, and whether the opponent has passed, and applies a fixed priority ordering of six rules to select an action. The first rule is an anti-overbid check: if the opponent has already passed and the agent is ahead, it passes rather than wasting cards. The second rule covers survival and endgame situations: when the agent must win the current round or is in the final round, it either protects a lead by playing its cheapest card or attempts to beat the deficit using the smallest card that suffices, falling back to its highest card if no winning card exists. The third rule abandons rounds where the deficit exceeds 10 points, preserving cards for later rounds where they may be more important. The fourth rule handles tied scores at the opening of a round: the agent samples uniformly at random from mid-range cards (values 3–7), which is the only random element in the policy. The fifth rule handles the case where the agent is behind but the round is still winnable: it plays the smallest card that overtops the opponent's

score, with an additional check that prevents spending face cards (J, Q, K) to win in round 1. If no such card exists, it passes. The sixth rule simply passes whenever the agent is already ahead. BaselineBot's demonstrations are used to pre-train the RL agent in Phase 3.

One surprising finding: across all 135 training generations, the RL agent's win rate against SmartBot and its win rate against RandomBot were not meaningfully different across generations. SmartBot's structured strategy created no extra difficulty for the RL agent compared to pure random play, showing that SmartBot's strategy was not very good.

4 Methods

4.1 Proximal Policy Optimization (PPO)

All agents are trained using Proximal Policy Optimization (PPO), an on-policy reinforcement learning algorithm. At a high level, PPO operates in a repeated cycle: the agent plays a batch of games using its current strategy, collects the resulting (state, action, reward) trajectories, and uses them to update the strategy. The core idea is to push the strategy toward actions that led to better-than-expected outcomes and away from actions that led to worse-than-expected outcomes, but with a constraint that the update cannot change the strategy too drastically in any single step. This constraint is enforced by clipping the ratio between the new and old action probabilities to a narrow range around 1, which prevents large, destabilizing updates.

Alongside the strategy, PPO trains a value function, which is a separate estimate of how much total reward the agent can expect from any given game state. The value function is used to compute advantage estimates: for each action taken, the advantage measures whether the outcome was better or worse than the value function predicted for that state. This separates the credit for a good or bad outcome from the randomness of the game itself.

Entropy regularization can be applied to add a small bonus for keeping the agent's strategy unpredictable. Without this, the agent tends to always pick the same action in a given situation, which makes it easy for an opponent to anticipate.

$$L(\theta) = E[\min(r_t A_t, \text{clip}(r_t, 1 - e, 1 + e)A_t)]$$

The formula above is PPO's core update. r_t measures how much the new strategy differs from the old one for a given action. A_t measures how much better or worse that action turned out compared to what was expected. The $\text{clip}(\dots)$ keeps the update within a safe range. Advantages are estimated using Generalized Advantage Estimation (GAE), which balances accuracy and stability by blending short-term and long-term outcome signals. The implementation uses Stable-Baselines3's PPO with PettingZoo's multi-agent framework across 8 parallel game instances.

4.2 Direct PPO vs SmartBot

The first approach was straightforward: train the PPO agent by having it play thousands of games against SmartBot. This worked, as the agent reliably learned to beat SmartBot across multiple training runs (versions v1 through v7 with increasing training budgets). However, this did not lead to adequate generalization. We found that the resulting agent often made nonsensical moves and was unable to perform against human opponents. Because SmartBot

always plays the same way, the agent essentially memorized its patterns and capitalized on deficits in its algorithm. When tested against other opponents, performance was much weaker.

4.3 Ladder / Self-Play Training

To fix the problem of training against only one opponent, we switched to a self-play ladder. The idea was to train the agent against the previous version of itself, then use the newly trained agent as the opponent for the next round of training. This creates a kind of natural progression, as the agent gets better, so does its training partner.

Each round of training (called a generation) runs for 100,000 game steps. The training was bidirectionally split between playing as Player 1 and Player 2 (to avoid learning strategies that only work going first), mostly against the previous generation (70%) and a fixed early PPO baseline (30%). Every generation is automatically promoted to become the next training opponent regardless of how well it did, and this keeps the ladder always moving forward. After each generation we test against all four baseline opponents and log the results. We ran this for 135 total generations.

4.4 Multi-Opponent Sampling

After noticing that ladder training caused the agent to get worse at beating simpler opponents over time, we tried a variation: instead of training against only the immediately previous generation, each new generation would train against 10 randomly chosen opponents from the last 100 generations. The hope was that exposure to a variety of past opponents would keep the agent more generally capable.

In practice this did not work well. Training against 10 very different opponents at once made it hard for the agent to learn consistently, as the right strategy against a generation-5 opponent is quite different from the right strategy against a generation-90 opponent, and the agent struggled to do both at once. Older, weaker opponents also provided little useful feedback once the agent had gotten much stronger. We stopped pursuing this approach.

4.5 Behavioral Cloning + PPO from Demonstrations

Starting from a random initialization means the RL agent has to discover good strategies entirely through random trial and error, which takes a very long time. This third approach speeds things up by first teaching the agent to imitate an expert, then using RL to improve beyond that starting point.

Phase 1 -- Behavioral Cloning (BC). We record thousands of BaselineBot games in pairs of (game state, legal moves, chosen move). The agent is trained on these like a supervised learning problem: given this game state, predict what BaselineBot would do. One detail: when BaselineBot plays as Player 2, the game state is described from Player 2's perspective, but we want all training examples in the same format. We apply a state inversion, swapping the card and score data for the two players, so all examples look like they are from the agent's point of view. Training stops once the agent correctly predicts BaselineBot's move 75% of the time. Going higher risks memorizing moves rather than learning the strategy behind them.

Phase 2 -- PPO from Demonstrations (PPO-fD). The pre-trained agent then continues training through regular RL against BaselineBot. We add an extra term to the loss that keeps penalizing the agent whenever it disagrees with what BaselineBot would have done in the same situation:

$$L_{total} = L_{PPO} + \lambda L_{BC} + \alpha L_{value} - \beta H(\pi)$$

L_{PPO} is the standard RL signal. L_{BC} is the imitation penalty (weighted by $\lambda = 0.1$), which pulls the agent back toward BaselineBot's decisions whenever it drifts too far. L_{value} trains the value estimate on all transitions. $H(\pi)$ is the entropy bonus to keep the strategy varied. Together, these let the agent improve beyond BaselineBot while not forgetting what it learned from watching it.

5 Results and Analysis

5.1 Ladder Training Dynamics

Figure 1 shows how win rates against all four opponents changed over 135 training generations, smoothed with a 5-generation rolling average.

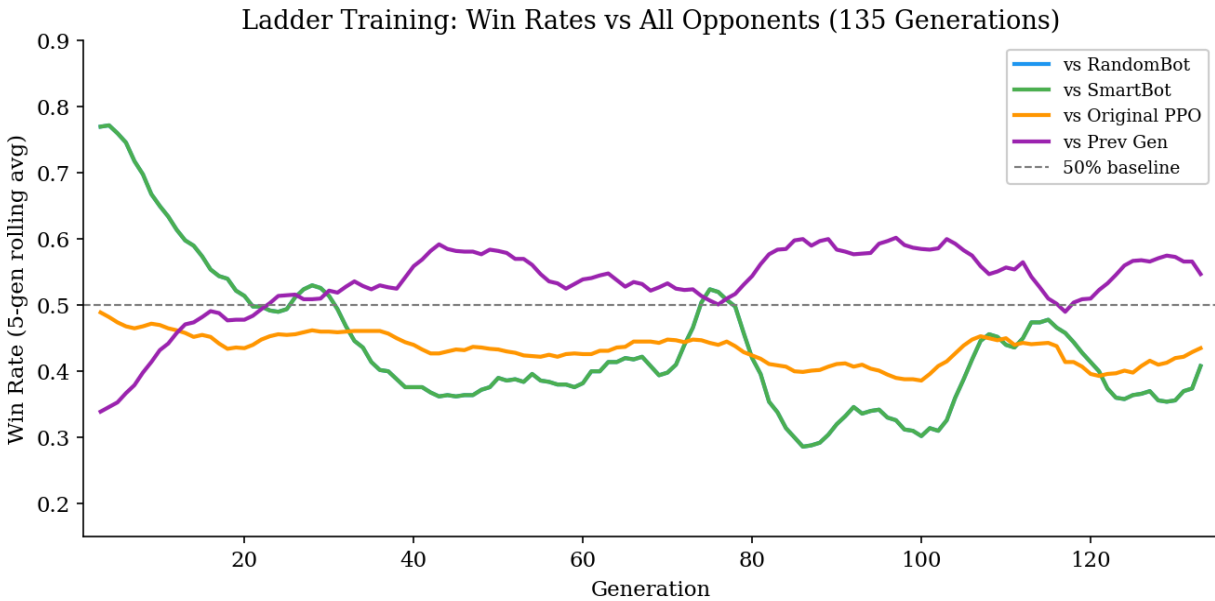


Figure 1. Win rates vs. RandomBot, SmartBot, original PPO, and the previous generation over 135 ladder training generations (5-generation rolling average). RandomBot and SmartBot win rates are identical throughout.

At generation 1, the agent wins about 77% of games against both RandomBot and SmartBot. By generation 100 that drops to around 26-30%. At the same time, the win rate against the previous generation climbs from 33.5% to 55%. The agent gets better at beating the opponent it just trained against, but progressively worse at beating simpler opponents it used to handle easily. This trade-off is the central pattern of the experiment.

5.2 Specialization Trade-off

Figure 2 shows this trade-off directly. The left panel shows the growing gap between the win rate against RandomBot (going down) and against the previous generation (going up). The right panel shows the overall average win rate with 95% confidence intervals.

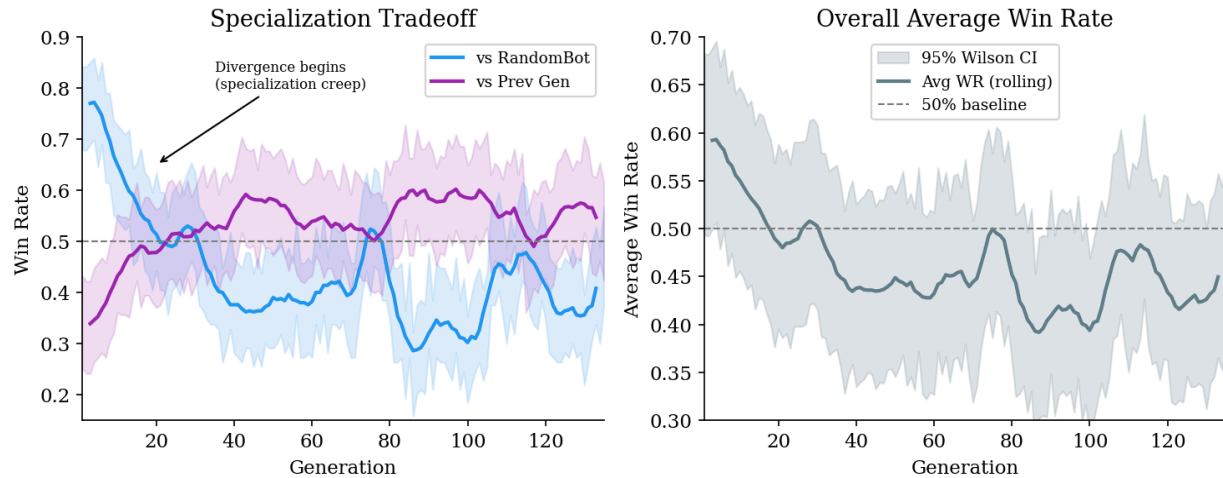


Figure 2. Left: divergence between RandomBot win rate (general ability) and previous-generation win rate (ladder progress) over 135 generations. Specialization is visible from around generation 10. Right: overall average win rate with 95% confidence intervals.

The average win rate starts at 59% and peaks at 60.5% by generation 4. It then falls to a low of 35.5% at generation 86, and partially recovers afterward. After generation 20 it hovers between 42-50% with no clear upward trend. The agent is not getting broadly better, it is just shifting what it is good at. Each new generation is harder to beat than the last, but in becoming specialized to beat that one opponent, it loses ground against everyone else. The system ends up cycling rather than converging on a generally strong strategy.

The RandomBot = SmartBot finding adds to this picture. Even SmartBot's structured rules gave the agent no extra challenge compared to random play. The inherent randomness of card draws is large enough that differences in opponent skill level do not create reliably different outcomes at the win-rate level.

5.3 BC + PPO-fD: Preliminary Results

Figure 3 shows the win rate against BaselineBot during the PPO-fD training phase.

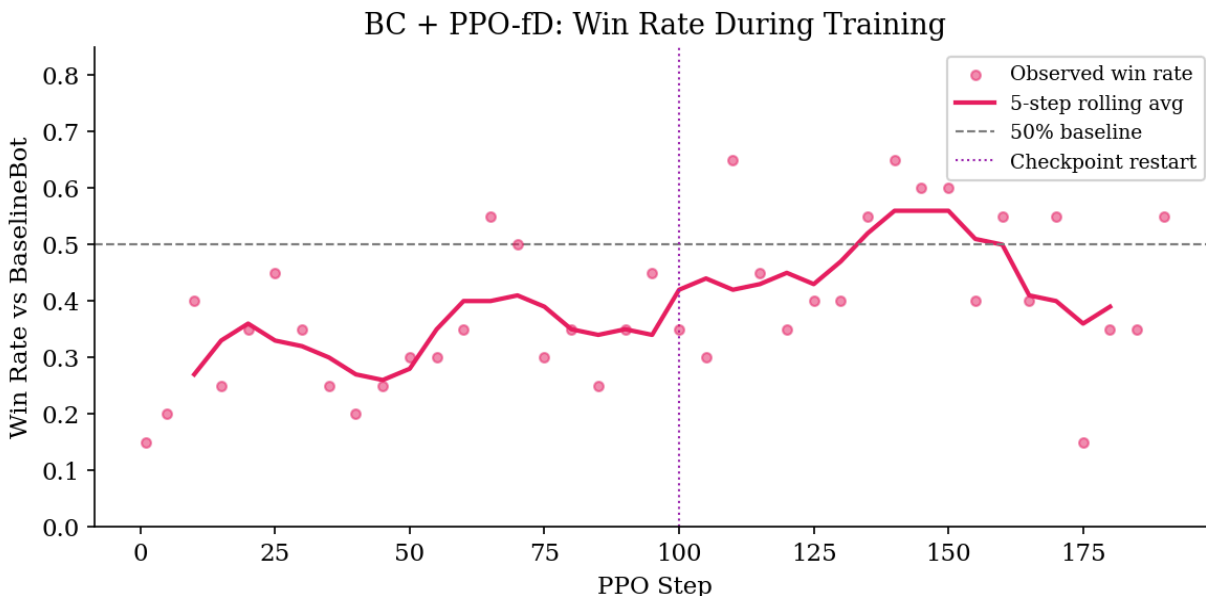


Figure 3. Win rate vs. BaselineBot during BC + PPO-fD training over 500 steps. Results are highly variable with no stable upward trend. The imitation loss (L_{BC}) reads as 0.0 throughout, due to a bug in the rollout buffer (see text).

The BC pre-training phase only reached 11.5% accuracy, far below the 75% target, because we only collected 1,000 demonstration games instead of the recommended 10,000. The RL training phase that followed shows win rates ranging all over the place (15% to 70%) with no clear trend, averaging around 45%.

The imitation loss L_{BC} reads exactly 0.0 throughout all 500 training steps. This means the imitation signal was never used during training. There is a bug where the rollout buffer was not recording BaselineBot's moves during games, so there were no bot examples to learn from. The agent was effectively just running standard PPO from a weak starting point with no imitation guidance at all. Despite this, it still averaged around 45% against BaselineBot, which is competitive with the ladder training results. This suggests that the BC warm start, even at low accuracy, gave the agent a slightly better starting point than training from scratch would have.

6 Discussion and Conclusion

This project tried three different approaches to training an RL agent on a two-player card game. Each one fixed a problem with the previous, but also introduced new challenges.

Training against a single fixed opponent (SmartBot) worked, but the agent only learned to beat that one opponent. Ladder self-play fixed this by always giving the agent a tougher opponent to train against, but created a new problem: the agent kept getting better at beating whoever it just trained against while getting progressively worse against everything else. We tried training against a random mix of past opponents to add variety, but the inconsistency of that training made it too hard for the agent to learn anything reliable.

The behavioral cloning + PPO approach is the most promising direction. Teaching the agent to imitate a known good strategy first, then improving it with RL, is a sensible way to avoid starting from nothing. The current implementation has a bug, as the imitation loss was never actually

active during RL training, but the underlying design is correct and the fix is clear: make the rollout buffer correctly record what BaselineBot did during each game.

Once that is fixed, the next step is to collect 10,000 BaselineBot demonstration games and retrain BC to the 75% accuracy target. Beyond that, the most robust solution to the specialization problem would be to keep a diverse pool of many past agents and train against a mix of them throughout training, rather than only the most recent one. Combining that with a proper BC warm start represents the strongest direction we see for future work.

More broadly, this project shows how difficult it is to train an agent that plays well in general, not just against a specific opponent. Self-play is a natural approach, but without careful design it tends to produce agents that are strong in a narrow sense and weak everywhere else. Building RL agents that can adapt to opponents they have never seen remains the challenge in this kind of work.